

10.6 Updating and downdating

A fitted regression often has to be changed slightly: new observations arrive, a suspicious case is set aside, a window slides along a time series, a variable is added or removed. Refitting from scratch costs $O(np^2)$ each time. Modifying the factorization costs $O(p^2)$ for a change of one row, and $O(np)$ for a change of one column. This section develops those modifications and uses them to compute every leave-one-out quantity from a single fit.

10.6.1 Adding and deleting observations

Adding a row is part (c) of Theorem 10.3.4: p rotations fold the new row into the triangle. Removing a row is harder, because orthogonal transformations can only combine rows that are present. The trick is to run the addition backwards.

Theorem 10.6.1 — Updating and downdating a triangular factor

Let $\mathbf{T}$ be an $m \times m$ nonsingular upper triangular matrix with $\mathbf{T}^\top \mathbf{T} = \mathbf{A}$, and let $\mathbf{z} \in \mathbb{R}^m$.

- (a) (*Adding a row.*) There are m rotations, the k th acting on row k of $\mathbf{T}$ and on an extra row, that turn $\begin{pmatrix} \mathbf{T} \\ \mathbf{z}^\top \end{pmatrix}$ into $\begin{pmatrix} \mathbf{T}_+ \\ \mathbf{0}^\top \end{pmatrix}$, with $\mathbf{T}_+$ upper triangular and $\mathbf{T}_+^\top \mathbf{T}_+ = \mathbf{A} + \mathbf{z}\mathbf{z}^\top$.
- (b) (*Deleting a row.*) Let $\mathbf{a}$ solve $\mathbf{T}^\top \mathbf{a} = \mathbf{z}$. The matrix $\mathbf{A} - \mathbf{z}\mathbf{z}^\top$ is positive definite iff $\|\mathbf{a}\| < 1$. In that case put $\alpha = (1 - \|\mathbf{a}\|^2)^{1/2}$. Choose rotations $\mathbf{G}_m, \dots, \mathbf{G}_1$, with $\mathbf{G}_k$ acting on coordinates k and $m+1$, such that $\mathbf{G}_1 \cdots \mathbf{G}_m (\mathbf{a}^\top, \alpha)^\top = (\mathbf{0}^\top, 1)^\top$. Then

$$\mathbf{G}_1 \cdots \mathbf{G}_m \begin{pmatrix} \mathbf{T} \\ \mathbf{0}^\top \end{pmatrix} = \begin{pmatrix} \mathbf{T}_- \\ \mathbf{z}^\top \end{pmatrix}$$

with $\mathbf{T}_-$ upper triangular and $\mathbf{T}_-^\top \mathbf{T}_- = \mathbf{A} - \mathbf{z}\mathbf{z}^\top$.

- (c) Each operation costs $O(m^2)$ flops: $3m^2$ for (a), and $3m^2$ plus a triangular solve of m^2 for (b).

Proof.

- (a) is Theorem 10.3.4(c).
- (b) $\mathbf{A} - \mathbf{z}\mathbf{z}^\top = \mathbf{T}^\top (\mathbf{I} - \mathbf{a}\mathbf{a}^\top) \mathbf{T}$. The matrix $\mathbf{I} - \mathbf{a}\mathbf{a}^\top$ has eigenvalue $1 - \|\mathbf{a}\|^2$ on $\mathbf{a}$ and 1 on $\mathbf{a}^\perp$. So it is positive definite iff $\|\mathbf{a}\| < 1$, and then so is $\mathbf{A} - \mathbf{z}\mathbf{z}^\top$ (Proposition 1.7.5(c)). The rotations exist: $\mathbf{G}_m$ combines a_m with α and leaves 0 in position m and $(a_m^2 + \alpha^2)^{1/2}$ in position $m+1$. Then $\mathbf{G}_{m-1}$ does the same with a_{m-1} , and so on (Theorem 10.3.4(a)). The final vector has norm $(\|\mathbf{a}\|^2 + \alpha^2)^{1/2} = 1$ and is zero except in the last position. That entry is positive, so it is 1. Apply the same rotations, in the same order, to $\begin{pmatrix} \mathbf{T} \\ \mathbf{0}^\top \end{pmatrix}$. Before $\mathbf{G}_k$ is applied, the extra row is zero in columns $1, \dots, k$, because it has only been mixed with rows $k+1, \dots, m$ of $\mathbf{T}$, which are zero there. Row k of $\mathbf{T}$ is zero in columns $1, \dots, k-1$. So $\mathbf{G}_k$ produces a new row k that is zero before column k , and the result is $\begin{pmatrix} \mathbf{T}_- \\ \mathbf{z}^\top \end{pmatrix}$.

with $\mathbf{T}_-$ upper triangular. Write $\mathbf{G} = \mathbf{G}_1 \cdots \mathbf{G}_m$, which is orthogonal. Then

$$\mathbf{z} = \mathbf{T}^\top \mathbf{a} = \begin{pmatrix} \mathbf{T} \\ \mathbf{0}^\top \end{pmatrix}^\top \begin{pmatrix} \mathbf{a} \\ \alpha \end{pmatrix} = \left(\mathbf{G} \begin{pmatrix} \mathbf{T} \\ \mathbf{0}^\top \end{pmatrix} \right)^\top \mathbf{G} \begin{pmatrix} \mathbf{a} \\ \alpha \end{pmatrix} = \begin{pmatrix} \mathbf{T}_- \\ \mathbf{w}^\top \end{pmatrix}^\top \begin{pmatrix} \mathbf{0} \\ 1 \end{pmatrix} = \mathbf{w},$$

and in the same way $\mathbf{T}^\top \mathbf{T} = \mathbf{T}_-^\top \mathbf{T}_- + \mathbf{w} \mathbf{w}^\top$. Hence $\mathbf{T}_-^\top \mathbf{T}_- = \mathbf{A} - \mathbf{z} \mathbf{z}^\top$.

(c) The rotations act on rows of length at most m , and there are m of them. ■

If $\mathbf{A} = \mathbf{X}^\top \mathbf{X}$ and $\mathbf{z} = \mathbf{x}_{(i)}$, then $\|\mathbf{a}\|^2 = \mathbf{z}^\top \mathbf{A}^{-1} \mathbf{z}$ is the leverage h_{ii} (Definition 6.8.1): a case can be deleted exactly when the others still determine every coefficient. Downdating the augmented triangle of $[\mathbf{X}, \mathbf{y}]$ (Proposition 10.2.2) gives the fit without case i , with the square root of the new SSE in the corner. Downdating is less stable than updating: $1 - \|\mathbf{a}\|^2$ cancels when $h_{ii} \approx 1$, and the relative error can reach about $u/(1 - h_{ii})$ (Stewart 1979).

```
import numpy as np
import statsmodels.api as sm

def givens(a, b):
    """c, s, r with [[c, s], [-s, c]] @ [a, b] = [r, 0]."""
    if b == 0.0:
        return 1.0, 0.0, a
    r = np.hypot(a, b)
    return a / r, b / r, r

def update(T, z):
    """Triangle of [A; z'] from the triangle T of A: rotate the new row into T."""
    T, z = T.copy(), np.array(z, dtype=float)
    for k in range(len(z)):
        c, s, r = givens(T[k, k], z[k])
        Tk, zk = T[k, k:].copy(), z[k:].copy()
        T[k, k:] = c * Tk + s * zk, -s * Tk + c * zk
    return T

def downdate(T, z):
    """Triangle of A with the row z' removed, from the triangle T of A (T'T = A'A)."""
    m = len(z)
    a = np.linalg.solve(T.T, z)          # T'a = z; ||a||^2 is the leverage of z
    alpha2 = 1.0 - a @ a
    if alpha2 <= 0:
        raise np.linalg.LinAlgError("the row cannot be removed (leverage >= 1)")
    alpha = np.sqrt(alpha2)
    T, w = T.copy(), np.zeros(m)        # w: the extra row, initially zero
    for k in range(m - 1, -1, -1):      # rotate (a_k, alpha) into (0, new alpha)
        c, s, r = givens(alpha, a[k])
        alpha = r
        Tk = T[k, k:].copy()
        T[k, k:], w[k:] = c * Tk - s * w[k:], s * Tk + c * w[k:]
    return T                             # on exit w equals z
```

[Run this code online](#)

The alternative, **recursive least squares** (Plackett 1950), updates $\mathbf{K} = (\mathbf{X}^\top \mathbf{X})^{-1}$ by the Sherman–Morrison formula (Theorem 1.6.5, Example (Deleting an observation)): adding $(\mathbf{x}, y)$ gives

$$\mathbf{K}_+ = \mathbf{K} - \frac{\mathbf{K} \mathbf{x} \mathbf{x}^\top \mathbf{K}}{1 + \mathbf{x}^\top \mathbf{K} \mathbf{x}}, \quad \hat{\boldsymbol{\beta}}_+ = \hat{\boldsymbol{\beta}} + \frac{\mathbf{K} \mathbf{x} (y - \mathbf{x}^\top \hat{\boldsymbol{\beta}})}{1 + \mathbf{x}^\top \mathbf{K} \mathbf{x}}.$$

It also costs $O(p^2)$, but it carries an explicit inverse of the cross-product matrix, with its κ^2 sensitivity, and nothing corrects its accumulated errors (Example (Ten-year regressions of consumption)).

10.6.2 Leave-one-out quantities from one fit

Many diagnostics ask what would happen if observation i were left out. Computing them by n separate fits costs $O(n^2 p^2)$, but all of them follow from one fit.

Proposition 10.6.2 — Deletion formulas

Let $\mathbf{X}$ have full column rank, let $h_{ii} < 1$, and let $\hat{\boldsymbol{\beta}}_{(i)}$ and $\text{SSE}_{(i)}$ denote the least squares estimate and residual sum of squares computed without observation i . Then

- (a) $\hat{\boldsymbol{\beta}} - \hat{\boldsymbol{\beta}}_{(i)} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_{(i)} \hat{\varepsilon}_i / (1 - h_{ii})$;
- (b) $y_i - \mathbf{x}_{(i)}^\top \hat{\boldsymbol{\beta}}_{(i)} = \hat{\varepsilon}_i / (1 - h_{ii})$;
- (c) $\text{SSE}_{(i)} = \text{SSE} - \hat{\varepsilon}_i^2 / (1 - h_{ii})$.

Proof.

- (a) is Exercise (B1), which follows from the deletion formula of Example (Deleting an observation).
- (b) Multiply (a) by $\mathbf{x}_{(i)}^\top$: $\mathbf{x}_{(i)}^\top \hat{\boldsymbol{\beta}}_{(i)} = \mathbf{x}_{(i)}^\top \hat{\boldsymbol{\beta}} - h_{ii} \hat{\varepsilon}_i / (1 - h_{ii})$, so $y_i - \mathbf{x}_{(i)}^\top \hat{\boldsymbol{\beta}}_{(i)} = \hat{\varepsilon}_i + h_{ii} \hat{\varepsilon}_i / (1 - h_{ii}) = \hat{\varepsilon}_i / (1 - h_{ii})$.
- (c) Put $g = \hat{\varepsilon}_i / (1 - h_{ii})$ and let $h_{ji} = \mathbf{x}_{(j)}^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_{(i)}$ be the entries of $\mathbf{M}$. By (a), $y_j - \mathbf{x}_{(j)}^\top \hat{\boldsymbol{\beta}}_{(i)} = \hat{\varepsilon}_j + h_{ji} g$ for every j , including $j = i$, where it gives g by (b). Summing squares over all j ,

$$\sum_j (\hat{\varepsilon}_j + h_{ji} g)^2 = \text{SSE} + 2g (\mathbf{M} \hat{\boldsymbol{\varepsilon}})_i + g^2 (\mathbf{M}^2)_{ii} = \text{SSE} + g^2 h_{ii},$$

because $\mathbf{M} \hat{\boldsymbol{\varepsilon}} = \mathbf{0}$ and $\mathbf{M}^2 = \mathbf{M}$. Removing the $j = i$ term, which is g^2 , leaves $\text{SSE}_{(i)} = \text{SSE} - g^2 (1 - h_{ii}) = \text{SSE} - \hat{\varepsilon}_i^2 / (1 - h_{ii})$. ■

With a QR factorization the leverages cost $O(np)$ (Proposition 10.3.3(c)), and the rest $O(n)$. This gives the PRESS residuals $\hat{\varepsilon}_i / (1 - h_{ii})$, whose sum of squares is leave-one-out cross-validation (Allen 1974; Theorem 29.3.2), and the deleted variances $s_{(i)}^2 = \text{SSE}_{(i)} / (n - p - 1)$ behind the externally studentized residuals $t_i = \hat{\varepsilon}_i / (s_{(i)} \sqrt{1 - h_{ii}})$ of Chapter 20 (Theorem 20.3.3).

Example. (Leaving out the District of Columbia)

Refit the state murder-rate regression with all 51 jurisdictions, including the District of Columbia, which [Chapter 6](#) set aside. The residual sum of squares is 142.53 and the PRESS statistic is 230.52. The largest PRESS residual belongs to the District of Columbia. Its ordinary residual is 4.475 and its leverage 0.506, so its prediction error when it is left out is $\hat{\varepsilon}_i/(1 - h_{ii}) = 9.066$. Its externally studentized residual is 4.28. By [Proposition 10.6.2\(c\)](#), deleting it reduces the residual sum of squares to 101.96, the value found for the 50 states in Example (Murder rates by the augmented factorization). The script confirms all 51 deletions three ways: by the formulas, by downdating the augmented triangle, and by refitting.

```
data = sm.datasets.statecrime.load_pandas().data           # all 50 states and DC
y = data["murder"].to_numpy()
X = np.column_stack([np.ones(len(y)), data[["poverty", "single", "urban"]]])
n, p = X.shape

Q, R = np.linalg.qr(X)
beta = np.linalg.solve(R, Q.T @ y)
e = y - X @ beta
sse = e @ e
h = np.sum(Q ** 2, axis=1)                                # leverages
press_resid = e / (1 - h)                                  # y_i minus its prediction
                    without case i
sse_loo = sse - e ** 2 / (1 - h)                           # SSE after deleting case i
s_loo = np.sqrt(sse_loo / (n - 1 - p))
t_ext = e / (s_loo * np.sqrt(1 - h))                       # externally studentized
                    residuals
i = int(np.argmax(np.abs(press_resid)))
print(f"largest PRESS residual: {data.index[i]}, e = {e[i]:.3f}, h = {h[i]:.3f}, "
      f" e/(1-h) = {press_resid[i]:.3f}, t = {t_ext[i]:.2f}")
print(f"PRESS = {np.sum(press_resid ** 2):.2f}, SSE = {sse:.2f}")
```

[Run this code online](#)

10.6.3 Adding and deleting variables

Changing the columns of $\mathbf{X}$ is a Gram–Schmidt step, or its reverse.

Proposition 10.6.3 — Adding a column

Let $\mathbf{X} = \mathbf{Q}_1\mathbf{R}$ be a thin QR factorization of a full-column-rank $\mathbf{X}$, and let $\mathbf{z} \notin \mathcal{C}(\mathbf{X})$. Put $\mathbf{r} = \mathbf{Q}_1^\top \mathbf{z}$, $\mathbf{w} = \mathbf{z} - \mathbf{Q}_1\mathbf{r}$, $\rho = \|\mathbf{w}\|$ and $\mathbf{q} = \mathbf{w}/\rho$. Then

$$[\mathbf{X}, \mathbf{z}] = [\mathbf{Q}_1, \mathbf{q}] \begin{pmatrix} \mathbf{R} & \mathbf{r} \\ \mathbf{0}^\top & \rho \end{pmatrix}$$

is a thin QR factorization. The residual sum of squares falls by $(\mathbf{q}^\top \mathbf{y})^2$ when $\mathbf{z}$ is added, and the coefficient of $\mathbf{z}$ in the enlarged model is $\mathbf{q}^\top \mathbf{y}/\rho = \mathbf{w}^\top \mathbf{y}/\mathbf{w}^\top \mathbf{w}$.

Proof. $\mathbf{w} = (\mathbf{I} - \mathbf{M})\mathbf{z} \neq \mathbf{0}$ because $\mathbf{z} \notin \mathcal{C}(\mathbf{X})$, and $\mathbf{q}$ is a unit vector orthogonal to $\mathcal{C}(\mathbf{Q}_1)$. The product of the factors is $[\mathbf{Q}_1\mathbf{R}, \mathbf{Q}_1\mathbf{r} + \rho\mathbf{q}] = [\mathbf{X}, \mathbf{z}]$. By Proposition 10.3.3(d), applied to the enlarged factorization, the new column's sequential sum of squares is the square of its coordinate $\mathbf{q}^\top \mathbf{y}$. The last row of the triangular system of Proposition 10.3.3(a) reads $\rho b_z = \mathbf{q}^\top \mathbf{y}$. This is also the Frisch–Waugh–Lovell coefficient $\mathbf{w}^\top \mathbf{y} / \mathbf{w}^\top \mathbf{w}$ of Theorem 6.6.2. ■

If $\mathbf{z}$ is nearly in $\mathcal{C}(\mathbf{X})$, the computed $\mathbf{w}$ loses orthogonality to $\mathbf{Q}_1$; repeating the step once restores it (Björck 1996, chapter 2). Deleting column j of $\mathbf{R}$ leaves an upper Hessenberg block that $p - j$ rotations restore to triangular form, at $O(p^2)$ cost (Exercise (B2)). So stepwise procedures move between models at $O(p^2)$ or $O(np)$ cost per step.

```
def add_column(Q1, R, x):
    """Thin QR of [X, x] from the thin QR of X (one Gram-Schmidt step, repeated once)."""
    r = Q1.T @ x
    w = x - Q1 @ r
    r2 = Q1.T @ w                                # reorthogonalize: guards against
    cancellation
    w, r = w - Q1 @ r2, r + r2
    rho = np.linalg.norm(w)
    Rn = np.block([[R, r[:, None]], [np.zeros((1, R.shape[1])), np.array([[rho]])]])
    return np.column_stack([Q1, w / rho]), Rn
```

[Run this code online](#)

10.6.4 A moving window

Example. (Ten-year regressions of consumption)

Quarterly US real consumption is regressed on an intercept, real disposable income, real investment, population and unemployment (the public-domain macroeconomic series of Example (A macroeconomic design)), in a window of 40 quarters that slides forward one quarter at a time. Each step adds one row and deletes one. Over the 163 steps, whose windows have condition numbers up to 2.9×10^6 , the fit is maintained in two ways. The first applies two Sherman–Morrison updates to $(\mathbf{X}^\top \mathbf{X})^{-1}$. The second updates and then downdates the augmented triangle. After each step both are compared with a fresh QR fit of the window (Figure 10.6.1). Inverse updating reaches a relative error of 6.2×10^{-7} and ends at 8.4×10^{-8} . Updating the triangle stays below 1.1×10^{-10} and ends at 3.6×10^{-11} . Both are cheap. Only one keeps QR's accuracy.

```
macro = sm.datasets.macrodta.load_pandas().data
yw = macro["realcons"].to_numpy()
Xw = np.column_stack([np.ones(len(yw)), macro[["realdpi", "realinv", "pop", "unemp"]]])
w = 40                                # ten years of quarterly data
print("kappa of the full design:", f"{np.linalg.cond(Xw):.2e}")
```

[Run this code online](#)

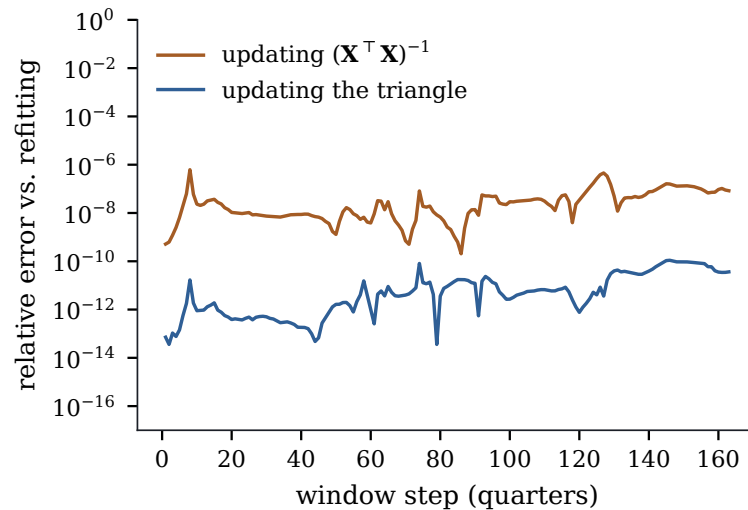


Figure 10.6.1: A moving-window regression maintained by rank-one updates of $(\mathbf{X}^T \mathbf{X})^{-1}$ and by Givens updating and downdating of the augmented triangle. The vertical axis is the relative difference in the coefficients from a fresh QR fit of each window.

```

K = np.linalg.inv(Xw[:w].T @ Xw[:w])          # (a) rank-one updates of the
inverse
g = Xw[:w].T @ yw[:w]
Zw = np.column_stack([Xw, yw])
Tw = np.linalg.qr(Zw[:w], mode="r")          # (b) triangle of [X, y]
p1 = Xw.shape[1]
err_sm, err_giv = [], []
for t in range(w, len(yw)):
    new, old = Xw[t], Xw[t - w]
    Kx = K @ new                             # add the new quarter (Sherman-
Morrison)
    K = K - np.outer(Kx, Kx) / (1 + new @ Kx)
    Kx = K @ old                             # delete the oldest quarter
    K = K + np.outer(Kx, Kx) / (1 - old @ Kx)
    g = g + new * yw[t] - old * yw[t - w]
    b_sm = K @ g
    Tw = downdate(update(Tw, Zw[t]), Zw[t - w])
    b_giv = np.linalg.solve(Tw[:p1, :p1], Tw[:p1, p1])
    b_ref = np.linalg.lstsq(Xw[t - w + 1:t + 1], yw[t - w + 1:t + 1], rcond=None)[0]
    # (c) refit
    err_sm.append(np.linalg.norm(b_sm - b_ref) / np.linalg.norm(b_ref))
    err_giv.append(np.linalg.norm(b_giv - b_ref) / np.linalg.norm(b_ref))
print(f"final relative error: inverse updating {err_sm[-1]:.1e}, Givens {err_giv
[-1]:.1e}")

```

[Run this code online](#)

Idea. A change of one row or column costs $O(p^2)$ or $O(np)$ on the triangular factor, not $O(np^2)$. All n leave-one-out fits cost one, and the leverage says whether, and how accurately, a case can be removed.

10.6.5 Exercises

10.6.5.1 A. Check your understanding

Exercise A1.

Show that $\text{PRESS} = \sum_i \hat{\varepsilon}_i^2 / (1 - h_{ii})^2 \geq \text{SSE}$. Show that if all leverages equal p/n , then $\text{PRESS} = \text{SSE} n^2 / (n - p)^2$. Evaluate this for the state data with $n = 51$ and $p = 4$, and compare with the actual ratio in [Example \(Leaving out the District of Columbia\)](#).

10.6.5.2 B. Practice

Exercise B1.

Show that the leverage of row i of the augmented matrix $\mathbf{Z} = [\mathbf{X}, \mathbf{y}]$ is $h_{ii} + \hat{\varepsilon}_i^2 / \text{SSE}$. Deduce the condition under which the augmented triangle can be downdated, and show that it fails only if $h_{ii} = 1$ or deleting case i leaves a model that fits the remaining data exactly.

Solution. $\mathcal{C}(\mathbf{Z}) = \mathcal{C}(\mathbf{X}) \oplus \text{span}(\hat{\mathbf{e}})$, with the two summands orthogonal, so by Theorem 6.3.7 the projection onto $\mathcal{C}(\mathbf{Z})$ is $\mathbf{M} + \hat{\mathbf{e}}\hat{\mathbf{e}}^\top / \|\hat{\mathbf{e}}\|^2$. Its i th diagonal entry is $h_{ii} + \hat{\varepsilon}_i^2 / \text{SSE}$. Downdating requires this to be below 1. By [Proposition 10.6.2\(c\)](#), $1 - h_{ii} - \hat{\varepsilon}_i^2 / \text{SSE} = (1 - h_{ii}) \text{SSE}_{(i)} / \text{SSE}$, which vanishes iff $h_{ii} = 1$ or $\text{SSE}_{(i)} = 0$.

Exercise B2.

Let $\mathbf{X} = \mathbf{Q}_1 \mathbf{R}$ and let $\mathbf{X}_{-j}$ be $\mathbf{X}$ without column j . Show that $\mathbf{X}_{-j} = \mathbf{Q}_1 \mathbf{H}$, where $\mathbf{H}$ is $\mathbf{R}$ without column j , which is upper Hessenberg in its last $p - j$ columns. Show that $p - j$ rotations of adjacent rows reduce $\mathbf{H}$ to $\begin{pmatrix} \mathbf{R}_{-j} \\ \mathbf{0}^\top \end{pmatrix}$, and that $\mathbf{R}_{-j}$ is the triangle of $\mathbf{X}_{-j}$. How much does the residual sum of squares rise?

Solution. Columns of $\mathbf{X}$ are $\mathbf{Q}_1$ times columns of $\mathbf{R}$, so deleting a column of both gives $\mathbf{X}_{-j} = \mathbf{Q}_1 \mathbf{H}$. Column $k \geq j$ of $\mathbf{H}$ is column $k + 1$ of $\mathbf{R}$, which has nonzeros down to row $k + 1$: one below the diagonal. Rotate rows $j, j + 1$ to zero entry $(j + 1, j)$. This mixes two rows that are zero before column j , so nothing earlier fills in. Continue with rows $(j + 1, j + 2)$ and so on. After $p - j$ rotations, $\mathbf{G}\mathbf{H} = \begin{pmatrix} \mathbf{R}_{-j} \\ \mathbf{0}^\top \end{pmatrix}$ with $\mathbf{R}_{-j}$ upper triangular, and $\mathbf{X}_{-j} = (\mathbf{Q}_1 \mathbf{G}^\top) \begin{pmatrix} \mathbf{R}_{-j} \\ \mathbf{0}^\top \end{pmatrix}$, where $\mathbf{Q}_1 \mathbf{G}^\top$ has orthonormal columns. Apply the same rotations to $\mathbf{c}_1 = \mathbf{Q}_1^\top \mathbf{y}$. The last entry of the rotated vector is the coordinate of $\mathbf{y}$ along the direction that has dropped out of the model, and its square is the rise in the residual sum of squares. This is the partial sum of squares of column j (Definition 9.3.2).

10.6.5.3 C. Going deeper

Exercise C1.

Feed the rows of $[\mathbf{X}, \mathbf{y}]$ one at a time into the augmented triangle, as in Theorem 10.3.4(c).

For $k > p$, let t_k be the entry left in the new row when case k is added. Show that

$$t_k = \pm \frac{y_k - \mathbf{x}_{(k)}^\top \hat{\boldsymbol{\beta}}_{k-1}}{(1 + \mathbf{x}_{(k)}^\top (\mathbf{X}_{k-1}^\top \mathbf{X}_{k-1})^{-1} \mathbf{x}_{(k)})^{1/2}},$$

where $\hat{\boldsymbol{\beta}}_{k-1}$ and $\mathbf{X}_{k-1}$ use the first $k-1$ cases. Show that under the normal linear model the t_k are independent $N(0, \sigma^2)$. These are the recursive residuals of Brown, Durbin and Evans (1975), used to test the constancy of a regression over time.